

Ride-matching paper

Ritun Saha

September 10, 2026

This project will develop the ride-pooling assignment problem as a sequential decision process under the influence of the four stated kinds of uncertainties and develop an optimal assignment policy with RL.

- travel time uncertainty
- service duration uncertainty at stops
- uncertain post-match booking cancellation from riders
- uncertain rider no-show

1 Some focus

- shared mobility optimization under the 4 stated uncertainty
- studies focus on drivers' behavior, such as relocation and order acceptance, but less attention is given to riders' behavior – an optimal strategy under riders' behavioral uncertainty is an opportunity
- offer acceptance is credited with a reward immediately, but uncertainty around trip completion is less explored

2 Question

We tentatively plan to address the following research questions in this study:

1. Can a learned policy trained under a system calibrated for uncertain operational conditions outperform benchmark deterministic heuristics?
2. Under no maximum radius for matching cap, what effective maximum pickup ETA does the RL policy tolerate before rejecting a request? How does this implicit threshold vary between different times of the day, different days, and different months? And how does it compare with the maximum allowable pickup ETA used by deterministic heuristics?
3. For the stop or passenger schedule violation, similar to Q2.
4. How does the learned policy handle extreme events or variability spikes, such as a sudden congestion spike?

3 Methods

3.1 State space

One episode is one day. The RL agent's decision epoch will be triggered per request arrival. The simulator will roll at a fixed time step of 5 seconds.

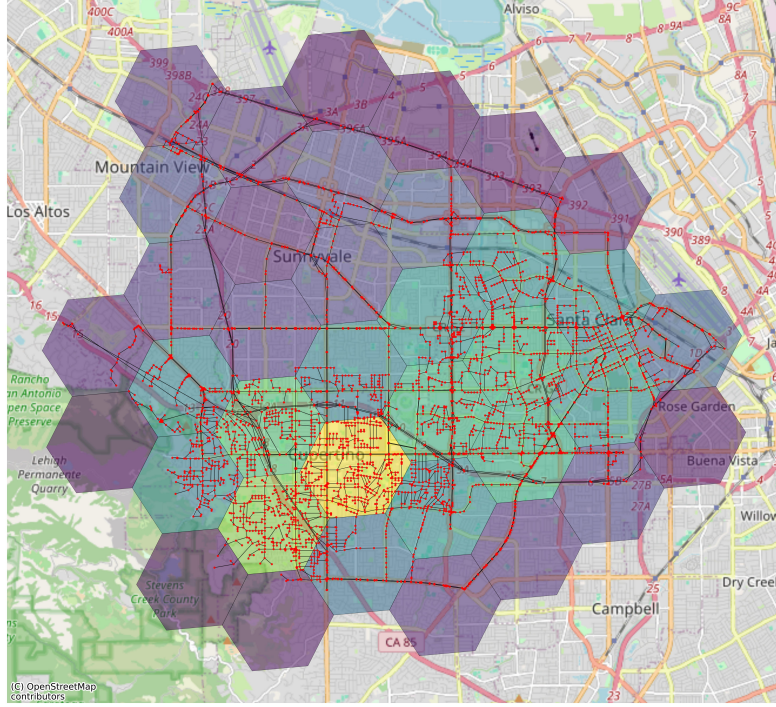


Figure 1: Hexagonal division of the service area

We will snap forward the request timestamp to be a multiple of 5 so that it coincides with one of the simulator’s time step boundaries (FleetPy already does it). In this way, we do not have to pause the simulator in the middle of a time step for feature extraction.

The agent will observe the state and make an assignment decision, which will then be forwarded to the simulator to process and continue rolling.

Now, one important question here is how we handle multiple request arrivals with the same time stamp – it doesn’t matter because epoch is defined on request arrival, requests will be attended in FIFO manner

3.1.1 New/incoming request features

- We will use lat/lon for pickup and drop-off and Uber’s H3 spatial indexing zoning system for other rolling stats.
- direct travel time
- time of day, day of week, and month
- number of passengers
- post-match cancellation rate in the last 15-20 minute window in the current request zone, plus the neighboring 6 zones
- rejection rate anomaly for the destination zone and the neighboring 6 zones for the last 15 min. period (optional)

3.1.2 Candidate vehicle features

Top K numbers of vehicles will be selected based on the insertion cost.

Each day is one episode. We have 11 months of demand files (we will devide it between training and evaluation sets). Each day will be replayed with several different seeds. We should have enough episodes to train. The agent should see the demand variation across the months.

The agent may learn something like accepting a request marginally.

We will set lenient values for operator max. wait time and operator detour time factor, so that it becomes easier for the insertions heuristic to find at least one vehicle for most of the incoming requests. The idea is that we want to leave the rejection decision to the RL agent and not let the insertion heuristic to resolve the rejection automatically just because it could not find any vehicle with a tight constraint on operator max. wait time and operator detour time factor.

- Vehicle location (lat/lon)
- residual passenger capacity in the vehicle – this is relevant to workload
- directionality score from immediate insertion with heuristics (-1: opposite direction; 0: idle; +1: same direction)
- ETA to current request pickup (and/or distance)
- ETA performance of the vehicle - inserting one new request in a vehicle deteriorates the ETA performance - so let the agent see how the vehicle is doing ETA wise for the already assigned passengers - implement some feature to communicate this - do not add anything from the diffusion model - it is a behavioral model, and the operator should have no idea about it.
- heuristic insertion cost
- number of already assigned but not served yet requests from the vehicle plan

3.1.3 Fleet/global context features

- fraction of fleet idle
- fraction of fleet with infeasible flag - a vehicle plan can become infeasible if:
 - latest pickup time violated
 - max trip time violated
 - latest arrival time violated
- number of request arrival per minute in the last 10-minute or 20-minute window – we can normalize this using a 95th percentile rate from the historical data
- fraction of riders that declined the offer after getting an initial quote from the operator
- fraction of passengers (number canceled/number assigned) canceled in the last 10-minute and/or 20-minute window - taking the fraction means it is already normalized. This "number assigned" is the riders who got an offer from the operator, accepted it, and are waiting for pickup
- mean/median of the differences between promised ETA and realized ETA when cancelled, for the post-match cancelled riders in the last 10-minute and/or 20-minute window
- mean/median of the differences between promised ETA and realized ETA for picked-up riders

For the case when multiple requests arrive with the same timestamps, the insertion heuristic should run right after an assignment, so that if the same vehicle becomes one of the candidates again, its updated plan is visible to the agent. After processing all the requests, the simulator will continue rolling again.

3.2 Action space

At each decision epoch, the action space is $K+1$ -dimensional:

1. accept the request for one of the K vehicles or reject (the reject option is always available for the agent to choose)

4 Reward

We will develop the reward function in two parts.

In our case, a ride request finalized for pick-up does not guarantee that the trip will be completed. There will be no immediate positive reward for accepting a request at an epoch.

There will be an immediate negative reward or penalty (w_{reject}) for rejecting a request at each epoch.

After the RL agent takes an action at epoch t , an immediate penalty will be charged if the request in that epoch is rejected. Then the simulator will roll until the next epoch $t + 1$. In this duration, all the pickups, post-match cancellations, rider declines, and no-shows will be recorded.

The reward/penalties are:

- For each picked-up request, the reward is $w_{pickup} - w_{wait} * (\text{realized wait time before pickup})$
- For each declined rider, the penalty is $w_{decline}$
- For each cancelled request, the penalty is w_{cancel}

So the reward at epoch t is:

$$R_t = -w_{reject} + \left[\sum_{\text{picked-up requests}} w_{pickup} - w_{wait}(\text{realized wait time before pickup}) \right] - w_{decline} * N_{declined} - w_{cancel} * N_{cancelled} \quad (1)$$

The important thing is that for a decision from the RL agent that accepted a request for pickup, the corresponding pickup or cancellation may land 10-20 epochs later than the epoch when it was accepted. The agent has to reason about this.

5 Misc. – Important

Remember that setting a lenient value of the “operator max wait time” (utilized by the insertion heuristic) would interfere with the rider-decline behavioral model. The rider-decline model utilizes the value for the slope of the acceptance probability curve. Using a higher value would flatten it and change the decline behavior. Consider using a separate parameter instead of “operator max wait time” for this model.